18

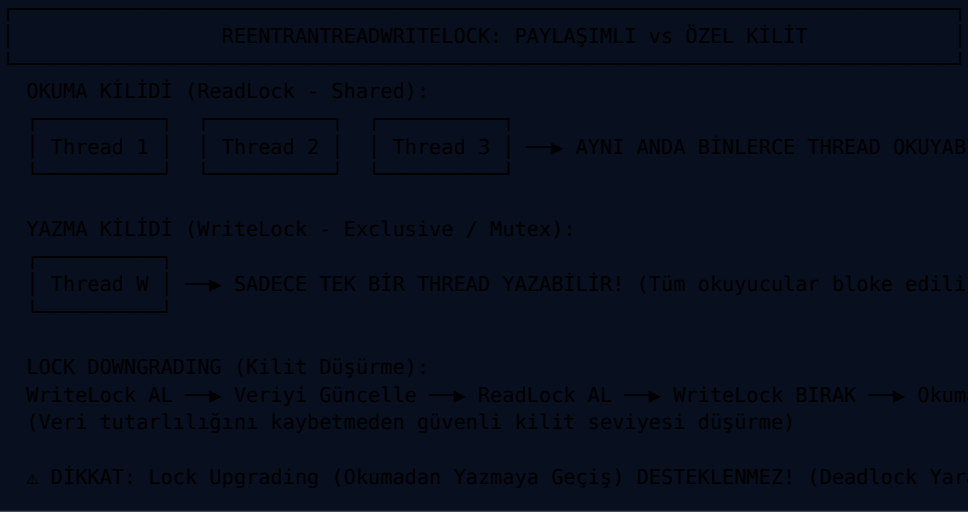
BÖLÜM: KLASIK SENKRONIZASYON, MONITÖRLER & KILITLER | MODULE: CLASSICAL SYNCHRONIZATION, MONITORS & LOCKS

ReentrantReadWriteLock: Okuma Yoğun Sistemler & Kilit Düşürme (Lock Downgrading)

ReentrantReadWriteLock: Read-Intensive Scaling & Lock Downgrading Mechanics

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kütüphanedeki büyük ansiklopedi düşünün. Aynı anda 100 öğrenci gelip ansiklopediyi sessizce okuyabilir (Okuma Kilidi / ReadLock); kimse kimseyi engellemez. Ama ansiklopedideki hatalı bir sayfayı yırtıp yenisini yapıştıracak olan matbaacı (Yazma Kilidi / WriteLock) geldiğinde, kütüphanedeki bütün öğrencileri dışarı çıkartır ve kapıyı kilitler! İşini bitirince kapıyı açar ve öğrenciler tekrar içeri dolar. Okuma çok, yazma azsa sistem roket gibi hızlanır!

🇬🇧 ENGLISH (ELI5)

Imagine a public stock ticker display. Thousands of traders can stare at the display concurrently without interfering with one another (Shared Read Lock). However, when the exchange operator updates the asset prices (Exclusive Write Lock), everyone must pause until the prices settle. If 99% of requests are reads and 1% are writes, a ReadWriteLock boosts concurrency a thousandfold compared to a blunt mutex!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

ReadWriteLock Mimarisi ve Kilit Düşürme Kuralları: 1) Durum Yönetimi (State Split): AQS'in 32-bitlik tek `state` tamsayısı ikiye bölünür: Üst 16 bit okuma kilidi sayısını (Shared count), alt 16 bit ise yazma kilidi sayısını (Exclusive count) tutar. 2) Lock Downgrading (Kilit Düşürme): Yazma kilidini tutarken önce okuma kilidi alınır, ardından yazma kilidi bırakılır. Bu sayede yazma ile okuma arasına başka bir yazıcının girmesi engellenerek veri tazeliği garanti edilir. 3) Lock Upgrading Neden Yoktur?: Eğer iki okuyucu thread aynı anda yazma kilidine yükselmek (upgrade) isterse, ikisi de birbirinin okuma kilidini bırakmasını bekler ve anında kilitlenir (Deadlock). Bu yüzden Java'da kilit yükseltme yasaktır!

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

ReadWriteLock Internals & Downgrade Protocols: 1) 32-Bit State Partitioning: AQS encodes dual locks within a single atomic integer: the high 16 bits record shared read acquisitions; the low 16 bits record exclusive write hold counts. 2) Lock Downgrade Sequence: While holding the write lock, acquire the read lock, then release the write lock. This preserves atomic continuity; no competing writer can slip in between the write release and read acquisition. 3) The Ban on Lock Upgrading: Upgrading from read lock to write lock is strictly disallowed. If Thread A and Thread B both hold read locks and attempt to upgrade, both freeze waiting for the other to release, creating an immediate deadlock.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Lock Downgrading (Kilit Düşürme) Deseni
import java.util.concurrent.locks.ReentrantReadWriteLock;

public class CacheWithLockDowngrade {
    private final ReentrantReadWriteLock rwLock = new ReentrantReadWriteLock();
    private String cachedData = "Initial Data";
    private boolean cacheValid = false;

    public void processData() {
        rwLock.readLock().lock();
        if (!cacheValid) {
            // Önce okuma kilidini bırakmak zorundayız!
            rwLock.readLock().unlock();

            // Yazma kilidini al
            rwLock.writeLock().lock();
            try {
                if (!cacheValid) { // Re-check
                    cachedData = "Fresh Data from DB";
                    cacheValid = true;
                }
                // ⚠ KİLİT DÜŞÜRME (LOCK DOWNGRADE) BURADA BAŞLAR:
                rwLock.readLock().lock(); // 1. Okuma kilidini AL
            } finally {
                rwLock.writeLock().unlock(); // 2. Yazma kilidini BIRAK (Kilit düşürüldü!)
            }
        }

        try {
            // Veriyi güvenle oku
            IO.println("İşlenen Veri: " + cachedData);
        } finally {
            rwLock.readLock().unlock(); // 3. Son okuma kilidini BIRAK
        }
    }

    void main() {
        new CacheWithLockDowngrade().processData();
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Mülakat Tuzağı: Mülakatçı 'ReentrantReadWriteLock kullanırken okuma kilidi içindeyken yazma kilidi alabilir miyiz (Lock Upgrading)?' diye sorar. Yanıt: 'HAYIR! Kilit yükseltme (Lock Upgrading) Deadlock riski nedeniyle Java'da desteklenmez; thread sonsuza dek kilitlenir. Ancak Kilit Düşürme (Lock Downgrading) tamamen desteklenir!'

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Classic MAANG Trap: 'Can a thread holding a read lock directly upgrade to a write lock?' Senior Answer: 'NO! Lock upgrading is strictly prohibited in Java to prevent deadlocks. Attempting to acquire a write lock while holding a read lock deadlocks the thread forever. However, Lock Downgrading (write to read) is fully supported.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- ReadWriteLock, okuma yoğun (%90+ okuma) sistemlerde devasa verim artışı sağlar.
- Kilit Düşürme (Downgrade: Write -> Read) güvenle desteklenir.
- Kilit Yükseltme (Upgrade: Read -> Write) deadlock nedeniyle yasaktır.

🎯 Key Takeaways for the Interview

- ReadWriteLock shines in read-heavy workloads (90%+ reads).
- Lock Downgrading (WriteLock -> ReadLock -> unlock WriteLock) is fully supported.
- Lock Upgrading is strictly prohibited due to inherent cyclic deadlocks.